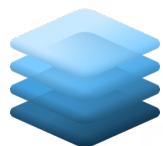


쉬운 도커

PART2. 이미지와 컨테이너



IMAGE

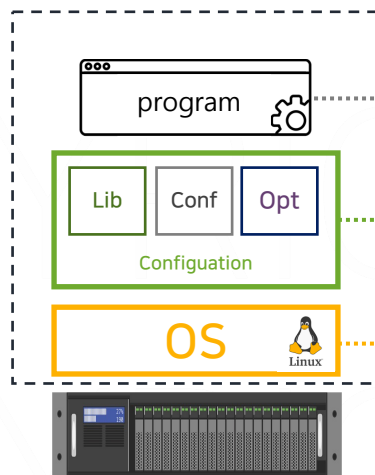


CONTAINER

PART2. 이미지와 컨테이너

이미지

웹서버 or
기타 소프트웨어 실행 서버

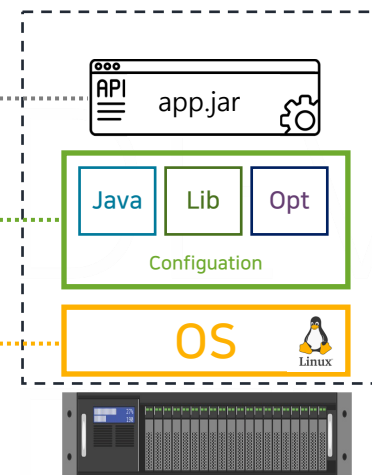


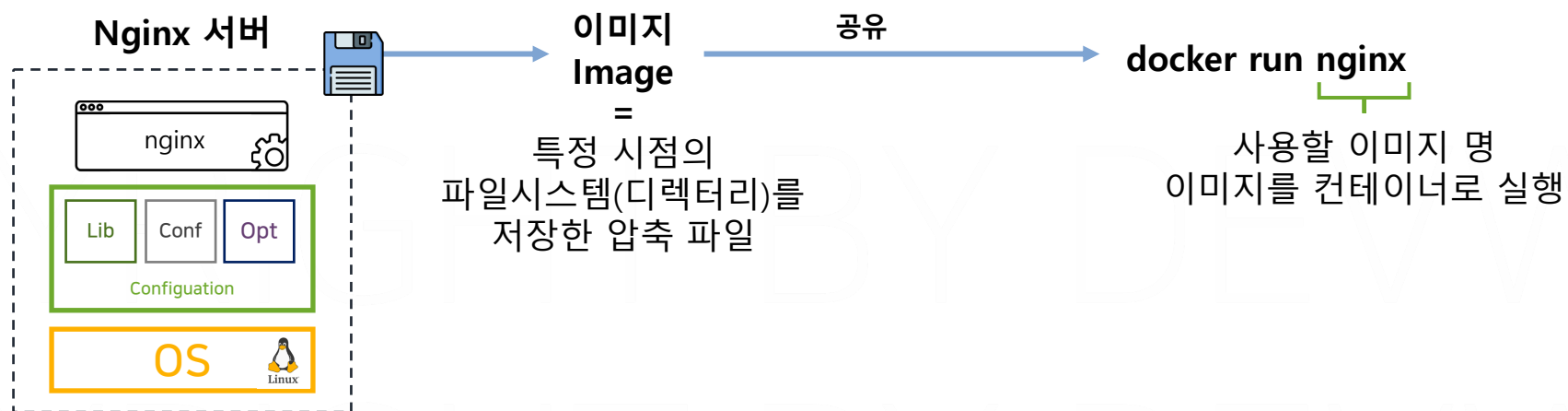
개발자가 개발한 애플리케이션 실행 파일
또는 다운로드 받은 소프트웨어

애플리케이션 실행에
필요한 언어, 라이브러리 및 설정 구성

애플리케이션 실행에
필요한 기본 시스템 파일

애플리케이션 서버





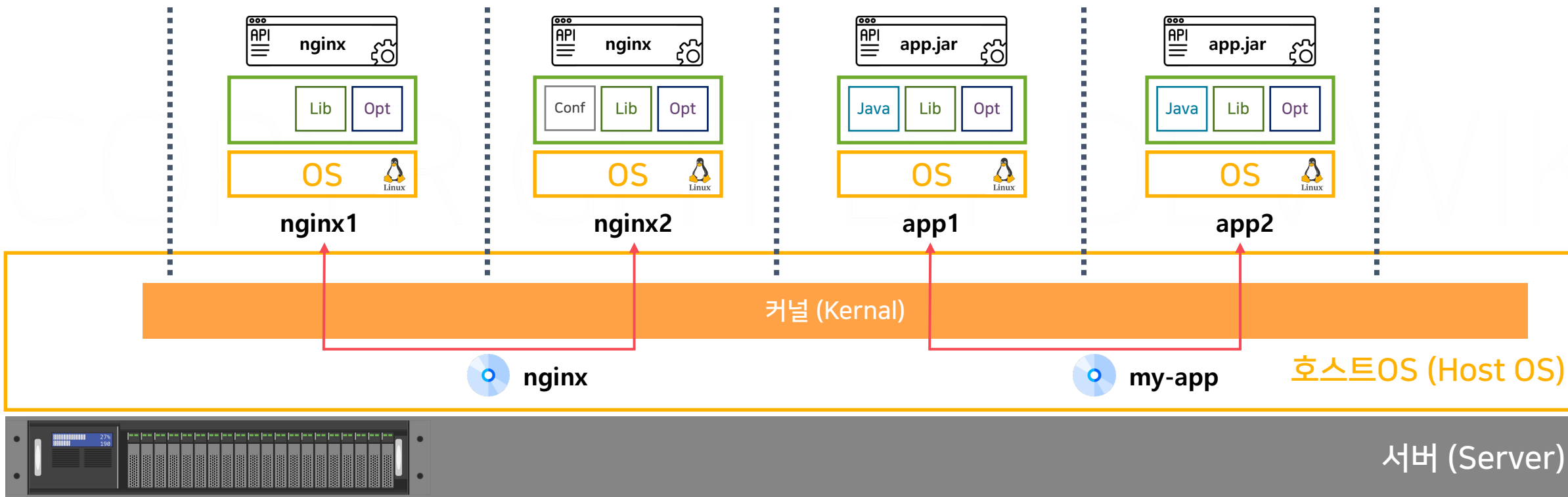
도커를 사용하는 목적: 가상화 서버를 빠르고 가볍게 운영하기 위해 사용

이미지: 특정 서버를 실행할 수 있는 상태를 저장한 압축 파일



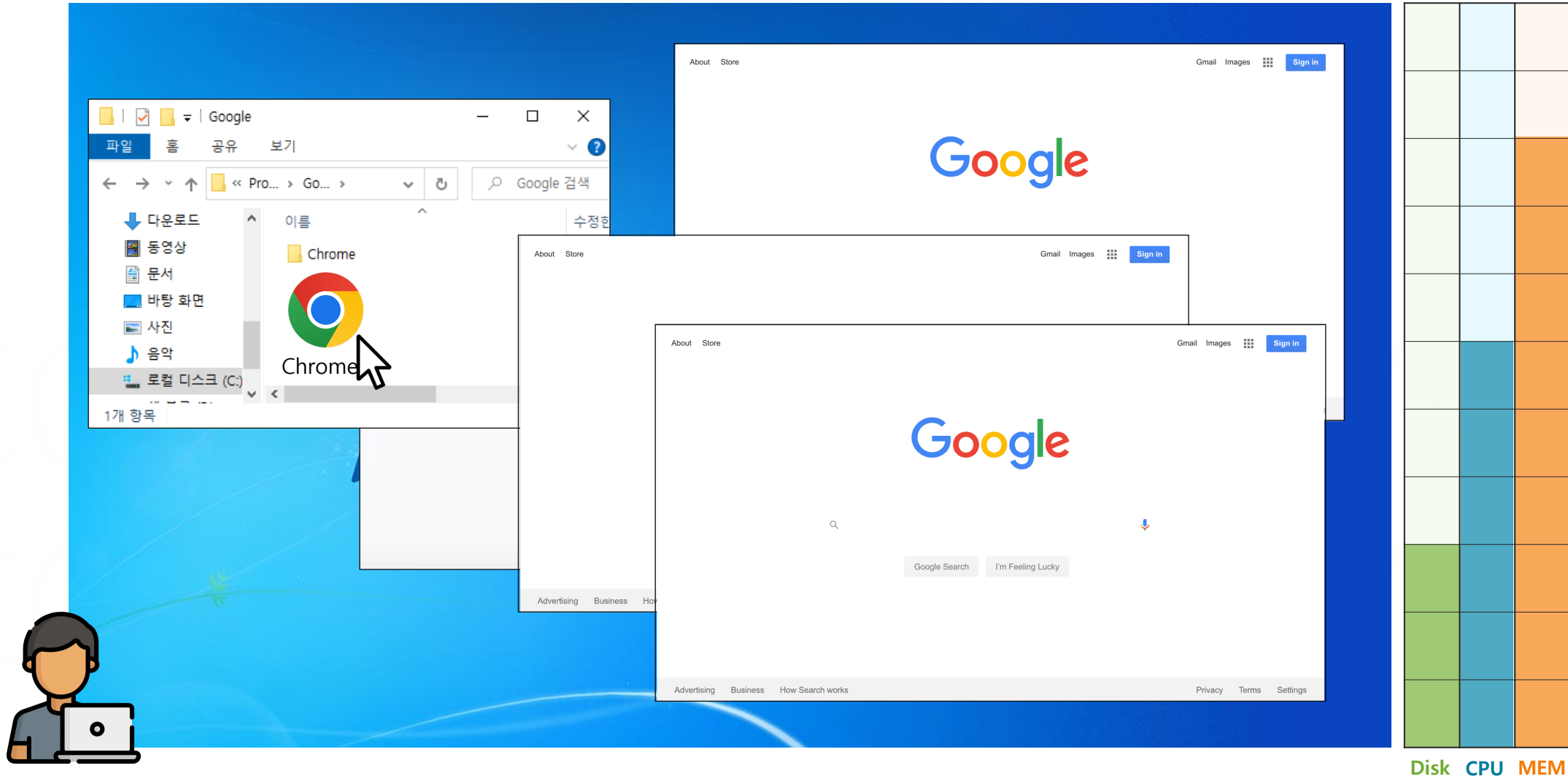
컨테이너 엔진

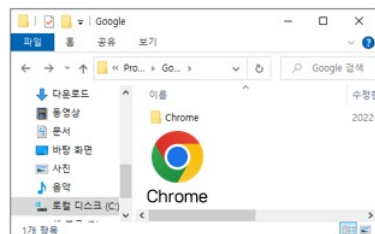
컨테이너 런타임



PART2. 이미지와 컨테이너

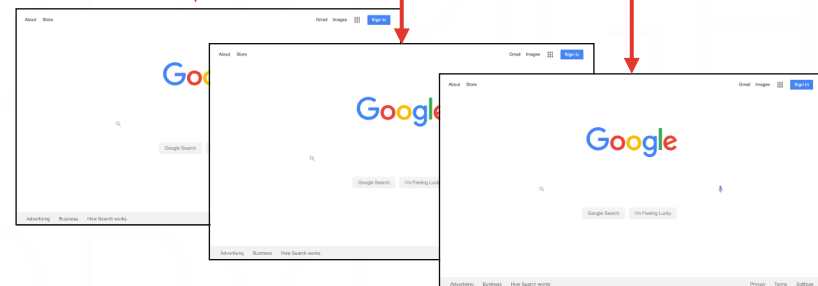
이미지와 컨테이너 실행





프로그램(Program)

- 실행 가능한 소프트웨어
- 디스크 공간 차지



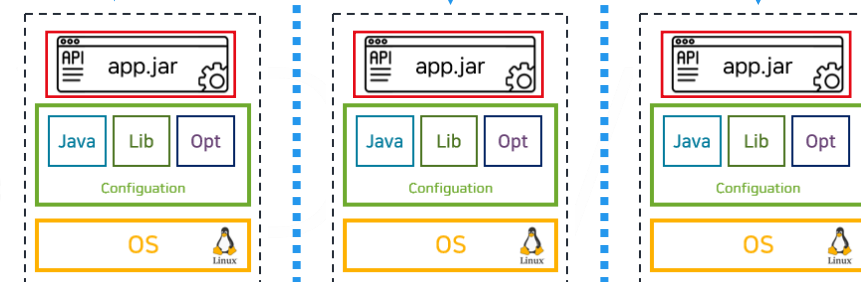
프로세스(Process)

- 실행 상태의 소프트웨어
- 프로그램 1 : N 프로세스
- 실행 시 CPU, MEM 등의 리소스 사용



이미지(Image)

- 실행 가능한 소프트웨어 + 실행에 필요한 환경
- 디스크 공간 차지



Docker

컨테이너(Container)

- 실행 상태의 이미지
- 이미지 1 : N 컨테이너
- 컨테이너로 실행 시 CPU, MEM 등의 리소스 사용
- 컨테이너 간 리소스 격리
- 실행 시 이미지를 복사해 격리된 디스크 공간 생성
- 컨테이너 실행 시 프로세스도 함께 실행

docker image ls (이미지명)

로컬 이미지 조회

이미지 목록, 단건 조회

1. 이미지 리스트 조회

```
docker image ls
```

2. nginx 이미지 조회

```
docker image ls nginx
```

COPYRIGHT BY DEVWIKI

COPYRIGHT BY DEVWIKI

백그라운드 실행

컨테이너의 이름 지정

docker run -d --name {컨테이너명} 이미지명

컨테이너 실행

docker ps

실행 중인 컨테이너 리스트 조회

docker rm -f

실행 중인 컨테이너 삭제

컨테이너 실행 및 조회

1. 이름이 다른 3개의 컨테이너 실행

```
docker run -d --name multinginx1 nginx
```

```
docker run -d --name multinginx2 nginx
```

```
docker run -d --name multinginx3 nginx
```

2. 실행 중인 컨테이너 리스트 조회

```
docker ps
```

```
🍏 ~/ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3b9b7ae61640	nginx	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	80/tcp	multinginx3
e0f8572ae316	nginx	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	80/tcp	multinginx2
028edb7b676e	nginx	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	80/tcp	multinginx1

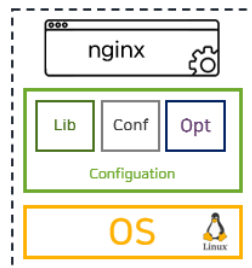
3. 다수의 컨테이너 삭제

```
docker rm -f multinginx1 multinginx2 multinginx3
```

PART2. 이미지와 컨테이너

메타데이터

이미지 파일



이미지의 메타데이터

Id: sha256:76c69f..
RepoTags: nginx: latest
Env: VERSION=1.23.2,PATH=/usr/..
Cmd: nginx -g daemon off;
Size: 14177820
...

nginx



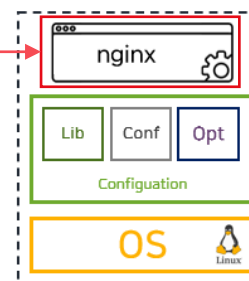
docker run

cmd 명령 실행

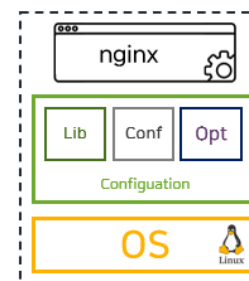
컨테이너의 메타데이터

Id: abc
RepoTags: nginx: latest
Env: VERSION=1.23.2,PATH=/usr/..
Cmd: nginx -g daemon off;
Size: 14177820
...

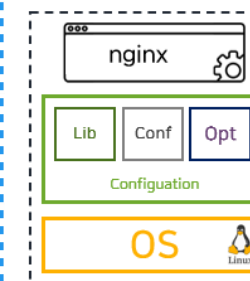
컨테이너 생성 시
컨테이너 메타데이터 복사



multinginx1



multinginx2



multinginx3

Docker

docker image inspect 이미지명

이미지의 세부 정보 조회

docker container inspect 컨테이너명

컨테이너의 세부 정보 조회

docker run 이미지명 (실행명령)

컨테이너 실행 시 메타데이터의 cmd 덮어쓰기

docker run --env KEY=VALUE 이미지명

컨테이너 실행 시 메타데이터의 env 덮어쓰기

컨테이너 실행 및 메타데이터 조회

1. nginx 이미지의 메타데이터 확인

```
docker image inspect nginx
```

2. 설정 변경사항 없이 컨테이너 실행

```
docker run -d --name defaultCmd nginx
```

3. 실행된 컨테이너의 메타데이터 확인

```
docker container inspect defaultCmd
```


메타데이터 덮어쓰기로 이미지의 파일 출력

1. docker run 명령의 옵션 확인

```
docker run --help
```

2. 메타데이터를 수정할 cmd 명령을 포함한 컨테이너 실행, cat 명령으로 컨테이너 내 파일 내용 출력

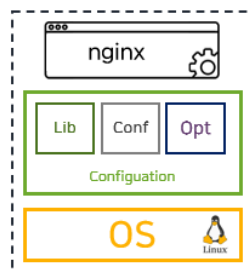
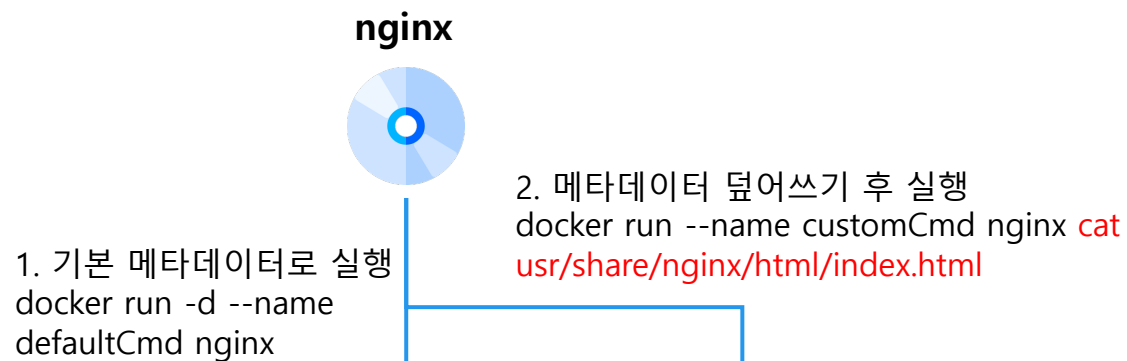
```
docker run --name customCmd nginx cat usr/share/nginx/html/index.html
```

3. 실행된 컨테이너의 메타데이터 확인

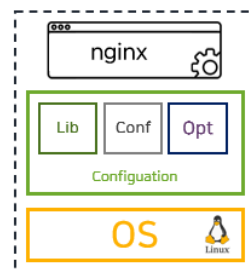
```
docker container inspect customCmd
```

4. docker ps -a 명령으로 종료된 컨테이너 조회

```
docker ps -a
```



defaultCmd



customCmd



```
[
  "Cmd": [
    "nginx",
    "-g",
    "daemon off;"
  ],

```

```
[
  "Cmd": [
    "cat",
    "/usr/share/nginx/html/index.html"
  ],

```

컨테이너를 백그라운드에서 실행

`docker run -d` 이미지명

-d 옵션 추가 시

지속적으로 실행되는
데몬 프로그램을 실행할 때 적합

-d 옵션 제거 시

실행 후 종료되는 프로그램에 적합,
실시간으로 로그를 확인할 경우

메타데이터 덮어쓰기로 애플리케이션 환경변수 수정

1. 이미지 다운로드

```
docker pull devwikirepo/envnodecolorapp
```

2. 이미지 메타데이터의 cmd, env 확인

```
docker image inspect devwikirepo/envnodecolorapp
```

3. 실행된 컨테이너의 메타데이터 확인, localhost:8080, 8081 접속

```
docker run -d -p 8080:3000 --name defaultColorApp devwikirepo/envnodecolorapp
```

```
docker run -d -p 8081:3000 --name blueColorApp --env COLOR=blue devwikirepo/envnodecolorapp
```

```
docker container inspect blueColorApp
```

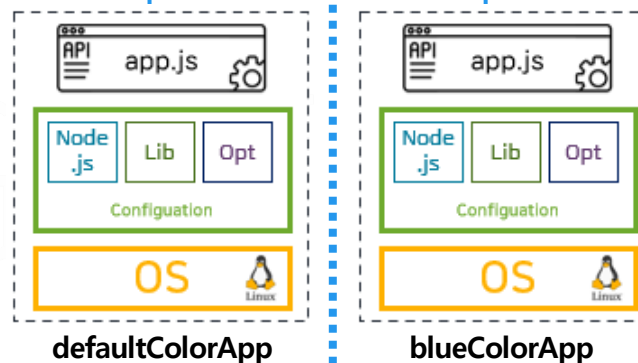
5. docker rm -f 로 실습에 생성한 컨테이너 모두 삭제

```
docker rm -f defaultCmd customCmd defaultColorApp blueColorApp
```

envnodecolorapp

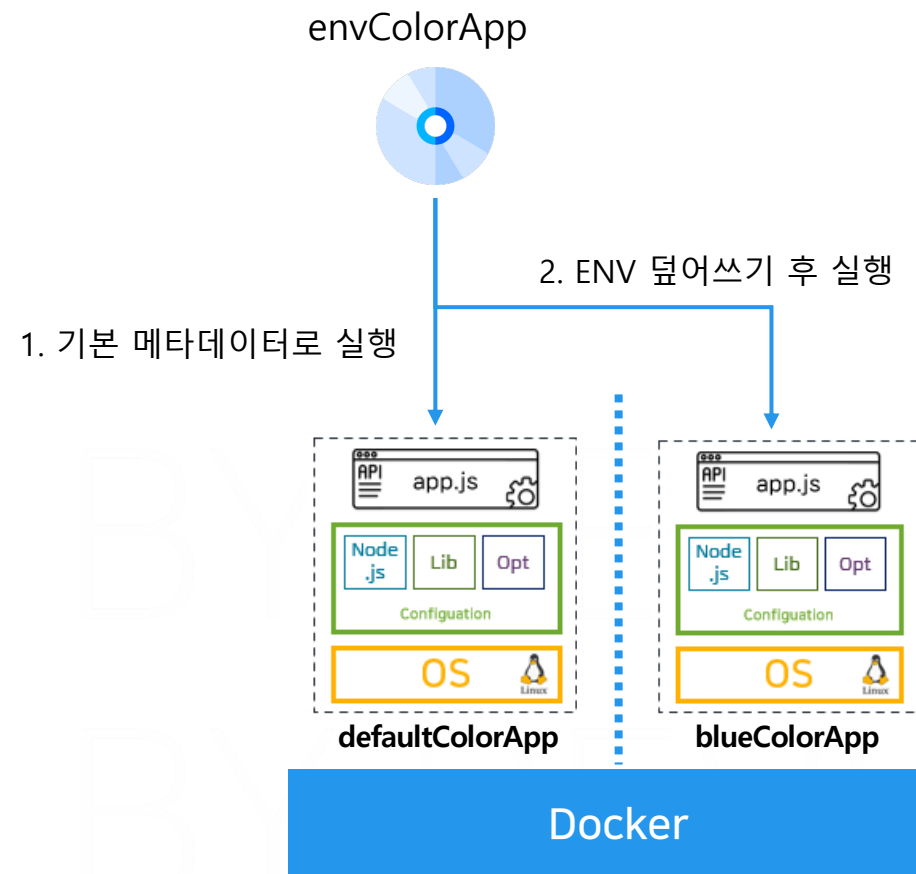
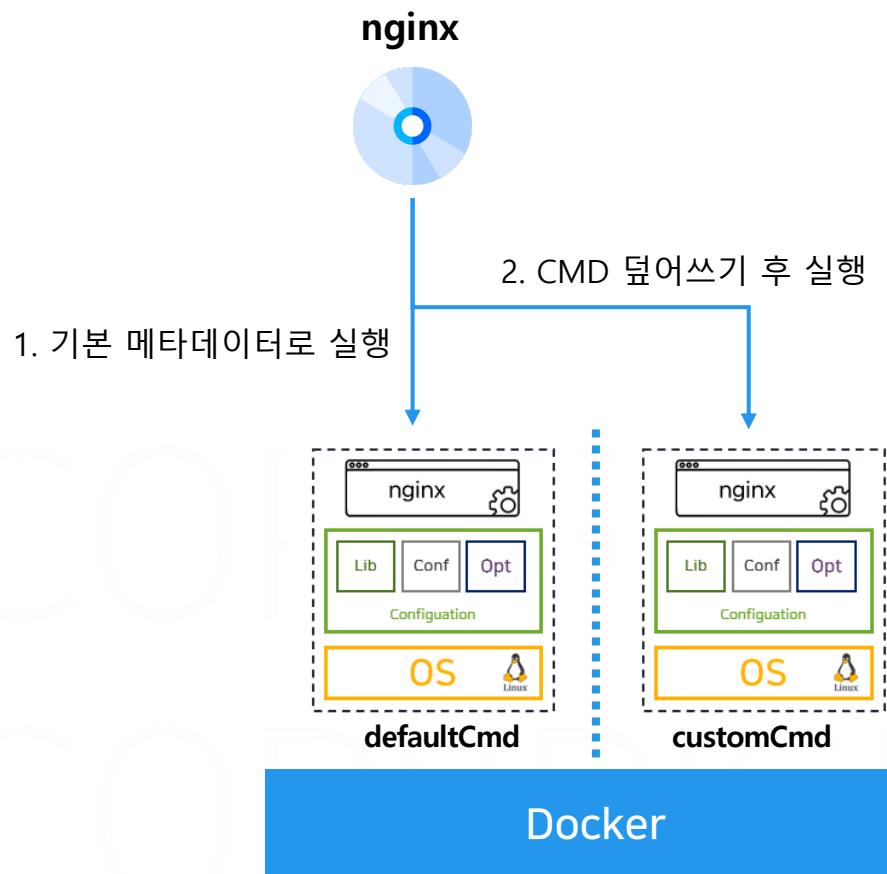
1. 기본 메타데이터로 실행
docker run -d --name
defaultColorApp
devwikirepo/envnodecolorapp

2. 메타데이터 ENV 추가 후 실행
docker run --name blueColorApp --env
COLOR=blue devwikirepo/envnodecolorapp



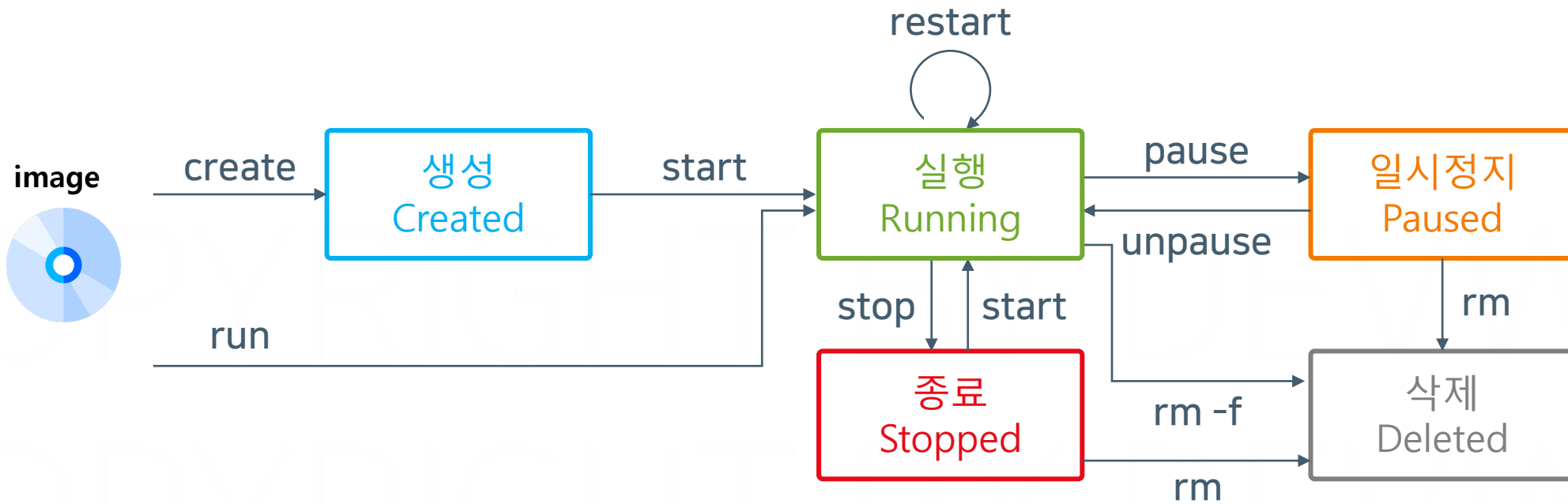
```
"Env": [
  "PATH=/usr/local/sbin:/u",
  "NODE_VERSION=14.21.3",
  "YARN_VERSION=1.22.19",
  "COLOR=red"
],
```

```
"Env": [
  "COLOR=blue",
  "PATH=/usr/local/sbin:/u",
  "NODE_VERSION=14.21.3",
  "YARN_VERSION=1.22.19"
],
```



PART2. 이미지와 컨테이너

컨테이너의 라이프사이클



docker logs (컨테이너명)

컨테이너의 로그 조회

컨테이너 생성 종료, 재시작

1. 컨테이너 생성

```
docker create --name tencounter devwikirepo/tencounter
```

2. 생성된 컨테이너 조회

```
docker ps -a
```

3. 컨테이너 실행

```
docker start tencounter
```

4. 컨테이너 실행 후 실행 및 종료 상태 확인

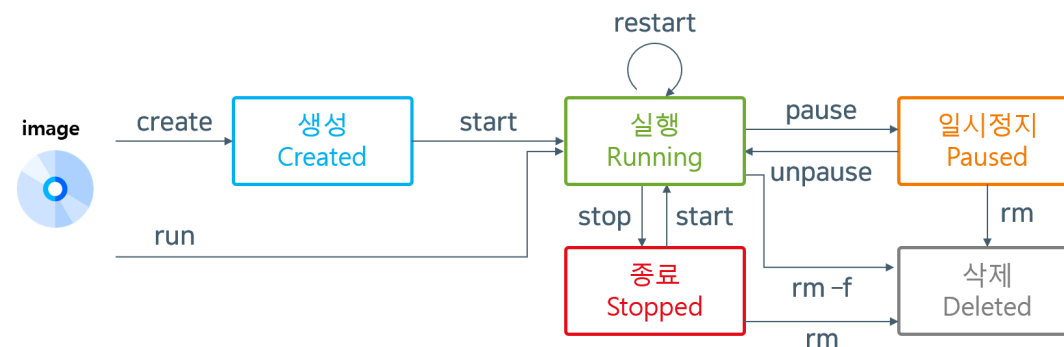
```
docker ps -a
```

5. 종료된 프로세스 재시작(터미널 연결)

```
docker start -i tencounter
```

6. 컨테이너 삭제

```
docker rm -f tencounter
```



컨테이너 실행 및 로그 확인(Shell1)

1. 컨테이너 실행

```
docker run --name hundredcounter devwikirepo/hundredcounter
```

Shell2로 이동 ==>

4. 컨테이너 재실행

```
docker start -i hundredcounter
```

Shell2로 이동 ==>

6. 컨테이너 상태 및 로그 확인

```
docker ps
```

```
docker logs hundredcounter
```

```
docker logs -f hundredcounter
```

7. 컨테이너 삭제

```
docker rm -f hundredcounter
```

==> 실습 종료

컨테이너 라이프사이클 조정(Shell2)

2. 컨테이너 중단 및 재시작

```
docker pause hundredcounter
```

```
docker unpause hundredcounter
```

3. 생성된 컨테이너 정지 후 상태 확인

```
docker stop hundredcounter
```

```
docker ps -a
```

<==== Shell1로 이동

5. 컨테이너 재시작

```
docker restart hundredcounter
```

<==== Shell1로 이동

